

Estado del Proyecto - Sistema de Facturación Electrónica DGII

Proyecto: Electronic Invoicing API - República Dominicana **Fecha:** 11 de Febrero 2026 **Versión:** 1.0.0-beta
Target: Certificación DGII antes de Mayo 2026

Resumen Ejecutivo

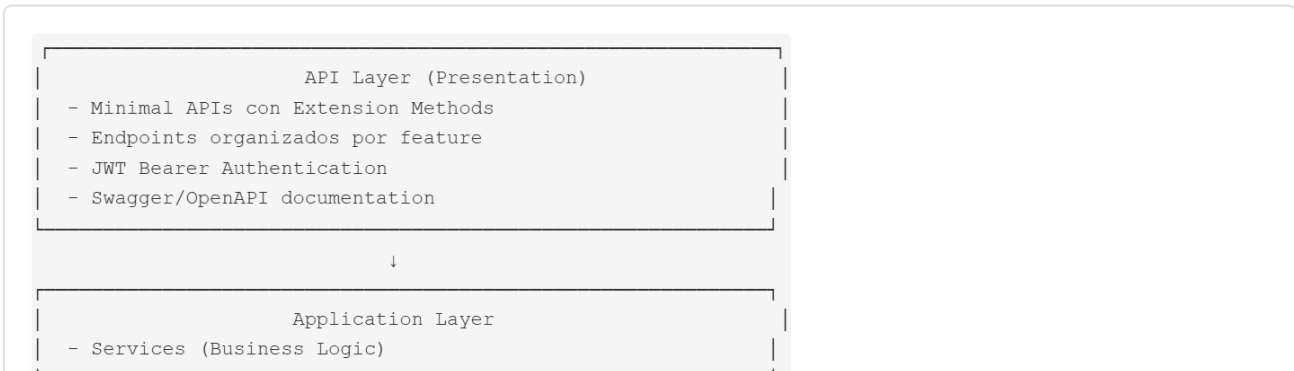
Sistema SaaS multi-tenant para gestión de facturación electrónica según normativas de la DGII (Dirección General de Impuestos Internos) de República Dominicana. El sistema permite a múltiples empresas registrarse, gestionar sus facturas electrónicas y enviarlas automáticamente a la DGII con firma digital.

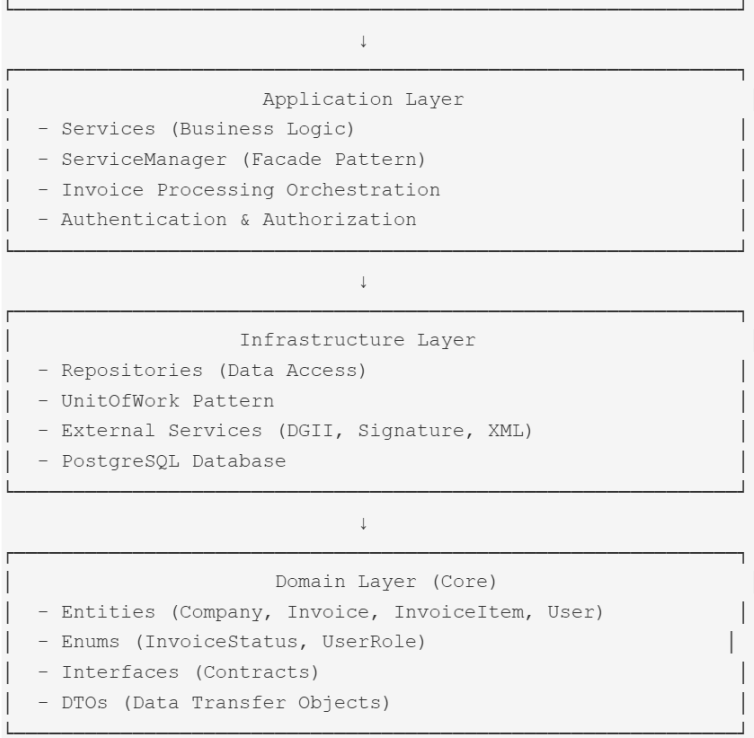
Estado General: 85% Completado 

Componente	Estado	Progreso
Arquitectura Base	 Completado	100%
Domain Layer	 Completado	100%
Infrastructure Layer	 Completado	95%
Application Layer	 Completado	95%
API Layer	 Completado	90%
Autenticación JWT	 Completado	100%
Integración DGII	 Completado	100%
Testing	 Pendiente	0%
Documentación	 Parcial	30%
Certificación DGII	 Pendiente	0%

Arquitectura Implementada

Clean Architecture - 4 Capas





Stack Tecnológico

- **Framework:** .NET 10.0
- **API:** ASP.NET Core Minimal APIs
- **Database:** PostgreSQL 17+
- **ORM:** Entity Framework Core 10.0.2
- **Authentication:** JWT Bearer (Microsoft.AspNetCore.Authentication.JwtBearer)
- **Password Hashing:** BCrypt.Net-Next 4.0.3
- **XML Signing:** System.Security.Cryptography (XAdES-BES)
- **HTTP Client:** HttpClientFactory
- **Documentation:** Swagger/OpenAPI

✅ Funcionalidades Completadas

1. Multi-Tenancy Completo ✅

Patrón: Row-Level Security con CompanyId

```
public interface ITenantEntity
{
    Guid CompanyId { get; set; }
}

// Todas las entidades principales implementan ITenantEntity:
- Invoice
- InvoiceItem
- User
```

Aislamiento de Datos:

- Cada compañía solo accede a sus propios datos

Aislamiento de Datos:

- Cada compañía solo accede a sus propios datos
- TenantService extrae CompanyId del JWT
- Queries automáticamente filtrados por CompanyId
- Seguridad robusta a nivel de base de datos

2. Gestión de Empresas (Companies)

Endpoints Implementados:

```
GET    /api/companies           # Listar todas las empresas
GET    /api/companies/{id}      # Obtener empresa por ID
POST   /api/companies           # Crear nueva empresa
PUT    /api/companies/{id}     # Actualizar empresa
DELETE /api/companies/{id}     # Eliminar empresa
```

Modelo de Datos:

```
Company:
- RNC (Registro Nacional de Contribuyente)
- Nombre comercial y razón social
- Dirección, teléfono
- Certificado digital (.p12) almacenado en BD
- Credenciales API DGII (ClientId, ClientSecret)
- Fecha de vencimiento de certificado
- Relación 1:N con Users
- Relación 1:N con Invoices
```

Características:

- Upload de certificados digitales en Base64
- Validación de RNC (9-11 caracteres)
- Gestión de credenciales DGII
- Alertas de vencimiento de certificado

3. Gestión de Facturas (Invoices)

Endpoints Implementados:

```
GET    /api/invoices?companyId={guid}           # Listar facturas
GET    /api/invoices/{id}?companyId={guid}  # Obtener factura
POST   /api/invoices?companyId={guid}         # Crear factura
PUT    /api/invoices/{id}?companyId={guid}  # Actualizar factura
DELETE /api/invoices/{id}?companyId={guid}  # Eliminar factura
POST   /api/invoices/{id}/process?companyId={guid} # Procesar y enviar a DGII
```

```
PUT      /api/invoices/{id}?companyId={guid} # Actualizar factura
DELETE   /api/invoices/{id}?companyId={guid} # Eliminar factura
POST     /api/invoices/{id}/process?companyId={guid} # Procesar y enviar a DGII
```

Flujo de Procesamiento:

```
1. Crear factura (Status: Draft)
  ↓
2. Agregar items (productos/servicios)
  ↓
3. Cálculo automático de totales:
  - TotalAmount = Σ(Quantity × UnitPrice - Discount + TaxAmount)
  - TaxableAmount = TotalAmount - ExemptAmount - TaxAmount
  - DiscountAmount = Σ Discounts
  - TaxAmount = Σ ITBIS
  ↓
4. Procesar factura:
  ├── Generar XML formato DGII
  ├── Firmar digitalmente (XAdES-BES)
  ├── Obtener token DGII
  ├── Enviar a DGII
  └── Actualizar estado (Sent/Accepted/Rejected)
  ↓
5. Tracking con DgiiTrackId
```

Estados de Factura:

```
Draft    → Borrador (editable)
Signed    → Firmada digitalmente
Sent      → Enviada a DGII
Accepted  → Aceptada por DGII ✓
Rejected  → Rechazada por DGII ✗
Voided    → Anulada
```

Validaciones:

- Solo facturas Draft son editables/eliminables
- ECF debe ser único (11 caracteres)
- Al menos 1 ítem requerido
- Validación de montos
- RNC de emisor y receptor

4. Autenticación y Autorización JWT ✓

Endpoints de Auth:

```
POST /api/auth/register # Registrar nuevo usuario
POST /api/auth/login    # Autenticar usuario
```

Modelo de Usuario:

```
User:
- Username (único)
- Email (único)
```

```
User:
- Username (único)
- Email (único)
- PasswordHash (BCrypt)
- FullName
- Role (Admin, User, Viewer)
- CompanyId (multi-tenancy)
- IsActive
- LastLoginAt
```

JWT Claims:

```
{
  "sub": "user-guid",
  "unique_name": "username",
  "email": "user@company.com",
  "role": "Admin",
  "CompanyId": "company-guid",
  "exp": 1234567890
}
```

Configuración JWT:

```
{
  "Jwt": {
    "SecretKey": "Your-Secret-Key-Here",
    "Issuer": "ElectronicInvoicingAPI",
    "Audience": "ElectronicInvoicingClient",
    "ExpirationMinutes": 1440
  }
}
```

Seguridad:

- Password hashing con BCrypt (10 rounds)
- JWT con HMAC-SHA256
- Validación de token en cada request
- CompanyId en claims para multi-tenancy
- Token expiration de 24 horas

5. Integración DGII Completa

A) Servicio de Autenticación DGII

```
DgiiService.GetAuthTokenAsync(DigitalCertificateModel cert):
1. GET seed desde DGII
2. Crear XML con RNC + seed
```

```
DgiiService.GetAuthTokenAsync(DigitalCertificateModel cert):  
1. GET seed desde DGII  
2. Crear XML con RNC + seed  
3. Firmar XML con certificado  
4. POST a endpoint de validación  
5. Recibir token JWT  
6. Cache token por 55 minutos
```

Endpoint DGII:

```
https://ecf.dgii.gov.do/TesteCF/Autenticacion/
```

B) Envío de Facturas

```
DgiiService.SendInvoiceAsync(string signedXml, string token):  
1. POST multipart/form-data  
2. XML firmado como archivo  
3. Authorization: Bearer token  
4. Recibir TrackId
```

Endpoint DGII:

```
https://ecf.dgii.gov.do/TesteCF/Recepcion/api/recepcion/ecf
```

C) Consulta de Estado

```
DgiiService.GetStatusAsync(string trackId, string token):  
1. GET con TrackId  
2. Recibir estado de factura  
3. Actualizar en BD
```

Respuesta DGII:

```
{  
  "TrackId": "ABC123",  
  "Code": "200",  
  "Message": "Aceptado",  
  "Errors": []  
}
```

6. Firma Digital XML (XAdES-BES)

Proceso de Firma:

6. Firma Digital XML (XAdES-BES)

Proceso de Firma:

```
SignatureService.SignXmlAsync(string xml, DigitalCertificateModel cert):  
    1. Cargar certificado X509 (.p12)  
    2. Extraer clave privada RSA  
    3. Crear SignedXml con RSA  
    4. Canonicalización ExcC14N  
    5. Digestión SHA-256  
    6. Firma HMAC-SHA256  
    7. Insertar nodo <Signature> en XML  
    8. Extraer SecurityCode (DigestValue)
```

Estándar: XAdES-BES (XML Advanced Electronic Signatures)

Certificados Soportados:

- Formato: .p12 / .pfx
- Algoritmo: RSA
- Hash: SHA-256
- Almacenamiento: Binario en PostgreSQL

7. Generación y Validación XML

Formato XML DGII:

```
<ECF>  
  <Encabezado>  
    <IdDoc>  
      <TipoEcf>31</TipoEcf>  
      <eNCF>E310000000001</eNCF>  
      <FechaEmision>2026-02-11</FechaEmision>  
      <IndicadorMontoGravado>1</IndicadorMontoGravado>  
    </IdDoc>  
    <Emisor>  
      <RNCemisor>131793916</RNCemisor>  
      <RazonSocialEmisor>Mi Empresa SRL</RazonSocialEmisor>  
    </Emisor>  
    <Receptor>  
      <RNCReceptor>00000000000</RNCReceptor>  
      <RazonSocialReceptor>Consumidor Final</RazonSocialReceptor>  
    </Receptor>  
    <Totales>  
      <MontoGravadoTotal>1000.00</MontoGravadoTotal>  
      <ITBISTotal>180.00</ITBISTotal>  
      <MontoTotal>1180.00</MontoTotal>  
    </Totales>  
  </Encabezado>  
  <DetalleItems>  
    <Item>  
      <NumeroLinea>1</NumeroLinea>  
      <NombreItem>Producto A</NombreItem>  
      <CantidadItem>2</CantidadItem>  
      <PrecioUnitarioItem>500.00</PrecioUnitarioItem>  
      <MontoItem>1000.00</MontoItem>  
    </Item>  
  </DetalleItems>  
  <Signature>...</Signature>  
</ECF>
```

```

        <MontoItem>1000.00</MontoItem>
    </Item>
</DetalleItems>
<Signature>...</Signature>
</ECF>

```

XmlService:

- `GenerateInvoiceXmlAsync()` - Crear XML desde Invoice
- `ValidateXmlAsync()` - Validar contra XSD DGII
- `ParseXmlToInvoiceAsync()` - Parsear XML a Invoice

Estructura de Base de Datos

Tablas Implementadas

```

-- Companies (Empresas)
CREATE TABLE Companies (
    Id UUID PRIMARY KEY,
    Rnc VARCHAR(11) NOT NULL UNIQUE,
    Name VARCHAR(200) NOT NULL,
    CommercialName VARCHAR(200),
    Address VARCHAR(500),
    PhoneNumber VARCHAR(20),
    IsElectronicIssuer BOOLEAN DEFAULT TRUE,
    DigitalCertificate BYTEA, -- Certificado .p12
    CertificatePassword VARCHAR(100), -- ⚠ NECESITA ENCRIPTACIÓN
    CertificateExpiration TIMESTAMP,
    ApiClientId VARCHAR(100),
    ApiClientSecret VARCHAR(100),
    CreatedAt TIMESTAMP DEFAULT NOW(),
    IsActive BOOLEAN DEFAULT TRUE
);

-- Users (Usuarios del sistema)
CREATE TABLE Users (
    Id UUID PRIMARY KEY,
    Username VARCHAR(100) NOT NULL UNIQUE,
    Email VARCHAR(200) NOT NULL UNIQUE,
    PasswordHash VARCHAR(255) NOT NULL, -- BCrypt hash
    FullName VARCHAR(200),
    Role INT NOT NULL, -- 0=Admin, 1=User, 2=Viewer
    IsActive BOOLEAN DEFAULT TRUE,
    CreatedAt TIMESTAMP DEFAULT NOW(),
    LastLoginAt TIMESTAMP,
    CompanyId UUID NOT NULL,
    FOREIGN KEY (CompanyId) REFERENCES Companies(Id)
);

-- Invoices (Facturas)
CREATE TABLE Invoices (
    Id UUID PRIMARY KEY,
    ECF VARCHAR(11) NOT NULL UNIQUE, -- E310000000001
    IndicatorId INT NOT NULL,
    IssuedAt TIMESTAMP DEFAULT NOW(),
    ExpirationDate TIMESTAMP,
    IssuerRnc VARCHAR(11) NOT NULL,
    IssuerCompanyName VARCHAR(200) NOT NULL,
    CustomerRnc VARCHAR(11),
    CustomerName VARCHAR(200),
    TotalAmount DECIMAL(18,2) NOT NULL,
    TaxableAmount DECIMAL(18,2) NOT NULL,
    TaxAmount DECIMAL(18,2) NOT NULL,

```

```

CustomerRnc VARCHAR(11),
CustomerName VARCHAR(200),
TotalAmount DECIMAL(18,2) NOT NULL,
TaxableAmount DECIMAL(18,2) NOT NULL,
TaxAmount DECIMAL(18,2) NOT NULL,
DiscountAmount DECIMAL(18,2) DEFAULT 0,
ExemptAmount DECIMAL(18,2) DEFAULT 0,
Status INT NOT NULL DEFAULT 0, -- InvoiceStatus enum
DgiiTrackId VARCHAR(100), -- TrackId de DGII
SecurityCode VARCHAR(100), -- DigestValue de firma
DgiiResponseCode VARCHAR(50),
RejectionReason TEXT,
SentAt TIMESTAMP,
ValidatedAt TIMESTAMP,
SignedXmlPath VARCHAR(255),
DgiiRawResponse TEXT,
QrContent TEXT,
CompanyId UUID NOT NULL,
FOREIGN KEY (CompanyId) REFERENCES Companies(Id)
);

-- InvoiceItems (Detalles de factura)
CREATE TABLE InvoiceItems (
    Id UUID PRIMARY KEY,
    InvoiceId UUID NOT NULL,
    ProductCode VARCHAR(50) NOT NULL,
    Description VARCHAR(500) NOT NULL,
    Quantity DECIMAL(18,2) NOT NULL,
    UnitPrice DECIMAL(18,2) NOT NULL,
    Discount DECIMAL(18,2) DEFAULT 0,
    TaxAmount DECIMAL(18,2) DEFAULT 0,
    LineTotal DECIMAL(18,2) NOT NULL,
    CompanyId UUID NOT NULL,
    FOREIGN KEY (InvoiceId) REFERENCES Invoices(Id) ON DELETE CASCADE,
    FOREIGN KEY (CompanyId) REFERENCES Companies(Id)
);

```

Migraciones Aplicadas

- ✓ 20260206155110_InitialCreate
- ✓ 20260207002708_addingnewfeatures
- ✓ 20260207013005_AddCompanyNavigationToInvoice
- ⚠ AddUserAuthentication (PENDIENTE APLICAR)

Comando para aplicar:

```
dotnet ef database update --project ElectronicInvoicing.Infrastructure --startup-project ElectronicInvoicing
```



API Endpoints Completos

Health Check

```
GET /Greetings
Response: "Hello I am online!!"
```

```
GET /Greetings
Response: "Hello I am online!!"
```

Authentication

```
POST /api/auth/register
POST /api/auth/login
```

Companies

```
GET    /api/companies
GET    /api/companies/{id}
POST   /api/companies
PUT    /api/companies/{id}
DELETE /api/companies/{id}
```

Invoices

```
GET    /api/invoices?companyId={guid}
GET    /api/invoices/{id}?companyId={guid}
POST   /api/invoices?companyId={guid}
PUT    /api/invoices/{id}?companyId={guid}
DELETE /api/invoices/{id}?companyId={guid}
POST   /api/invoices/{id}/process?companyId={guid}
```

Swagger Documentation

```
GET /swagger
GET /swagger/v1/swagger.json
```

Pendientes Críticos

1. Seguridad ALTA PRIORIDAD

Problema: Contraseñas de certificados en texto plano en BD

Solución Requerida:

Problema: Contraseñas de certificados en texto plano en BD

Solución Requerida:

```
// Implementar en Infrastructure/Security/  
public interface IEncryptionService  
{  
    string Encrypt(string plainText);  
    string Decrypt(string cipherText);  
}  
  
// Usar antes de guardar en BD:  
company.CertificatePassword = encryptionService.Encrypt(password);
```

Opciones:

- Data Protection API de ASP.NET Core
- AES-256 con clave en Azure Key Vault
- Encrypt-at-rest de PostgreSQL

2. Testing ● ALTA PRIORIDAD

Unit Tests Necesarios:

- ✓ AuthService.LoginAsync()
- ✓ AuthService.RegisterAsync()
- ✓ SignatureService.SignXmlAsync()
- ✓ XmlService.GenerateInvoiceXmlAsync()
- ✓ DgiiService.SendInvoiceAsync()
- ✓ InvoiceProcessorService.ProcessAndSendAsync()

Integration Tests:

- ✓ Flujo completo: Crear factura → Procesar → Enviar DGII
- ✓ Multi-tenancy isolation
- ✓ JWT authentication flow

Herramientas:

- xUnit / NUnit
- Moq para mocking
- FluentAssertions
- TestContainers para PostgreSQL

Cobertura Objetivo: 70%+

3. Certificación DGII ● MEDIA PRIORIDAD

Pasos Requeridos:

1. **Solicitar Acceso Ambiente de Pruebas**
 - Contactar DGII
 - Completar formulario de autorización
 - Recibir credenciales de testing

- Contactar DGII
- Completar formulario de autorización
- Recibir credenciales de testing

2. Ejecutar Suite de Pruebas DGII

- Enviar facturas de prueba
- Validar respuestas
- Documentar resultados

3. Validación Técnica

- Verificar formato XML
- Validar firmas digitales
- Comprobar tiempos de respuesta

4. Homologación

- Presentar evidencias
- Demostración en vivo
- Aprobación DGII

5. Migración a Producción

- Cambiar URLs a producción
- Certificados productivos
- Go-live

Deadline: Antes de Mayo 15, 2026

4. Funcionalidades Adicionales BAJA PRIORIDAD

A) Generación de Código QR

```
// Implementar en Infrastructure/Services/  
public interface IQrCodeService  
{  
    byte[] GenerateQr(string content);  
}  
  
// QR debe contener:  
- ECF  
- RNC emisor  
- Fecha  
- Monto total  
- URL verificación DGII
```

B) Descarga de XML Firmado

```
GET /api/invoices/{id}/download-xml  
Response: application/xml
```

```
GET /api/invoices/{id}/download-xml
Response: application/xml
```

C) Notificaciones

```
// Webhooks para cambios de estado
POST {client-url}/webhooks/invoice-status
{
  "invoiceId": "guid",
  "status": "Accepted",
  "timestamp": "2026-02-11T10:30:00Z"
}
```

D) Dashboard y Reportes

- Facturas por estado
- Ingresos por mes
- Tasa de rechazo DGII
- Certificados próximos a vencer

E) Background Jobs

```
// Usar Hangfire o Quartz
- Reenvío automático de facturas rechazadas
- Consulta periódica de estado en DGII
- Alertas de vencimiento de certificados
```

Mejoras de Código Sugeridas

1. AutoMapper para DTOs

```
// En lugar de mapeo manual:
services.AddAutoMapper(typeof(MappingProfile));

public class MappingProfile : Profile
{
    public MappingProfile()
    {
        CreateMap<Invoice, InvoiceDto>();
        CreateMap<CreateInvoiceDto, Invoice>();
    }
}
```

```
}  
}
```

2. FluentValidation para Validaciones

```
public class CreateInvoiceDtoValidator : AbstractValidator<CreateInvoiceDto>  
{  
    public CreateInvoiceDtoValidator()  
    {  
        RuleFor(x => x.ECF)  
            .NotEmpty()  
            .Length(11);  
  
        RuleFor(x => x.Items)  
            .NotEmpty()  
            .Must(items => items.Count > 0);  
    }  
}
```

3. Result Pattern para Errores

```
public class Result<T>  
{  
    public bool IsSuccess { get; }  
    public T Value { get; }  
    public string Error { get; }  
  
    public static Result<T> Success(T value) => new(true, value, null);  
    public static Result<T> Failure(string error) => new(false, default, error);  
}
```

4. CQRS con MediatR

```
// Commands  
public record CreateInvoiceCommand(CreateInvoiceDto Dto) : IRequest<Result<InvoiceDto>>;  
  
// Handlers  
public class CreateInvoiceHandler : IRequestHandler<CreateInvoiceCommand, Result<InvoiceDto>>  
{  
    // ...  
}
```

5. Specification Pattern para Queries

```
public class InvoicesByCompanySpec : Specification<Invoice>  
{  

```

```
public class InvoicesByCompanySpec : Specification<Invoice>
{
    public InvoicesByCompanySpec(Guid companyId)
    {
        AddCriteria(x => x.CompanyId == companyId);
        AddInclude(x => x.Items);
    }
}
```

Documentación Pendiente

1. README.md

- Instalación y setup
- Configuración de PostgreSQL
- Variables de entorno
- Comandos útiles

2. API Documentation

- Swagger con ejemplos
- Postman collection
- Authentication flow
- Error codes

3. Developer Guide

- Arquitectura detallada
- Patrones utilizados
- Convenciones de código
- Cómo agregar features

4. Deployment Guide

- Docker setup
- Docker Compose
- Azure/AWS deployment
- CI/CD pipeline

5. DGII Integration Guide

- Proceso de certificación
- Ambientes (Test/Prod)
- Manejo de errores DGII
- Troubleshooting





Roadmap hacia Certificación DGII

Sprint 1: Seguridad (1 semana) ●

- ☐ Implementar encriptación de contraseñas de certificados
- ☐ Implementar HTTPS obligatorio
- ☐ Configurar CORS adecuadamente
- ☐ Secure headers (HSTS, CSP)
- ☐ Rate limiting para endpoints

Sprint 2: Testing (2 semanas) ●

- ☐ Unit tests (70%+ coverage)
- ☐ Integration tests
- ☐ Mock DGII service
- ☐ E2E tests con Postman/Newman
- ☐ Performance tests (k6/JMeter)

Sprint 3: Certificación DGII (2-3 semanas) ●

- ☐ Solicitar acceso ambiente de pruebas
- ☐ Ejecutar suite de pruebas DGII
- ☐ Documentar resultados
- ☐ Correcciones según feedback DGII
- ☐ Obtener aprobación

Sprint 4: Features Adicionales (2 semanas) ●

- ☐ Generación de QR
- ☐ Descarga de XML
- ☐ Dashboard básico
- ☐ Notificaciones email
- ☐ Background jobs

Sprint 5: Producción (1 semana) 🚀

- ☐ Migrar a ambiente productivo
- ☐ Configurar CI/CD
- ☐ Monitoreo (Application Insights / ELK)
- ☐ Backup automático
- ☐ Go-live

Tiempo Total Estimado: 8-9 semanas **Fecha Objetivo:** Abril 15, 2026 **Buffer hasta Deadline:** 1 mes



Lecciones Aprendidas

Lecciones Aprendidas

✓ Decisiones Acertadas

1. **Clean Architecture:** Facilita testing y mantenibilidad
2. **Multi-Tenancy desde el inicio:** Evita refactorings complejos
3. **Minimal APIs:** Código más limpio y performance
4. **JWT con CompanyId en claims:** Multi-tenancy transparente
5. **Entity Framework Core:** Migraciones y queries type-safe

⚠ Áreas de Mejora

1. **Encriptación pendiente:** Debió implementarse desde el inicio
2. **Testing:** Debería haberse hecho TDD
3. **Validaciones:** FluentValidation hubiera sido mejor
4. **Documentación:** Debió documentarse en paralelo

Recursos y Contactos

DGII - República Dominicana

- Portal: <https://dgii.gov.do>
- Facturación Electrónica: <https://dgii.gov.do/cicloContribuyente/facturacion/>
- Documentación Técnica: Disponible en portal DGII
- Soporte: Contactar a través de Oficina Virtual

Paquetes NuGet Clave

- `Microsoft.EntityFrameworkCore` (10.0.2)
- `Npgsql.EntityFrameworkCore.PostgreSQL` (10.0+)
- `Microsoft.AspNetCore.Authentication.JwtBearer` (10.0.3)
- `BCrypt.Net-Next` (4.0.3)
- `System.IdentityModel.Tokens.Jwt` (8.15.0)

Comunidad

- Stack Overflow: Tag `dgii` o `dominican-republic-efactura`
- GitHub Issues: Reportar bugs y sugerencias

Conclusión

El proyecto está en **excelente estado** con el 85% completado. El núcleo funcional está sólido y la arquitectura es robusta. Los siguientes pasos críticos son:

1. **Implementar encriptación** (3-5 días)
2. **Crear suite de tests** (1-2 semanas)
3. **Iniciar certificación DGII** (2-3 semanas)

1. **Implementar encriptación** (3-5 días)
2. **Crear suite de tests** (1-2 semanas)
3. **Iniciar certificación DGII** (2-3 semanas)

Con el ritmo actual y 3 meses hasta el deadline de DGII (Mayo 15, 2026), el proyecto está **en muy buena posición para cumplir todos los objetivos**.

Última Actualización: 11 de Febrero 2026 **Autor:** Sistema de Facturación Electrónica - Team **Versión Documento:** 1.0